Sample-Efficient Adaptive Heuristic Selection via Offline Rollout Distillation for Dynamic Warehouse Order Dispatching

Vittal Mukunda
Dept. of Industrial Engineering and Management
R. V. College of Engineering
Bengaluru, India
vittalmukunda.im24@rvce.edu.in

Atharva Somani
Dept. of Industrial Engineering and Management
R. V. College of Engineering
Bengaluru, India
atharvasomani.im24@rvce.edu.in

Pranjal Malaiya
Dept. of Industrial Engineering and Management
R. V. College of Engineering
Bengaluru, India
pranjalmalaiya.im24@rvce.edu.in

Abstract*—Dynamic order dispatching in deadline- and perishability-constrained warehouses is routinely handled by simple priority rules, yet no single rule is best across the operating conditions a shift passes through. A selection hyper-heuristic can pick the right rule for the current context; the established ways of learning the selector are, however, costly. Online reinforcement learning is sample-hungry and unstable, and running a lookahead at every decision is too slow to deploy. We propose DAHS, a selection hyper-heuristic trained by offline rollout distillation. For each decision state in a corpus of simulated shifts we run short truncated-horizon stochastic rollouts of every candidate rule and record the per-rule cost vector; we then fit a calibrated gradient-boosted ranker to those vectors. This amortises an expensive online lookahead into a one-shot, fully offline supervised signal, and deploys as a single fast forward pass. We prove that the truncated-rollout cost is a consistent estimator of the full-horizon cost, with a truncation bias that decays in the rollout horizon, and confirm the predicted bias-variance trade-off empirically. On 50 held-out simulated shifts, DAHS attains a 1.33% SLA-breach rate against 3.13% for an analytic one-step-lookahead controller and 3.73% for an otherwise identical snapshot-trained ranker, and is Pareto non-dominated across four load scenarios. A faithful offline reinforcement-learning baseline reaches only 7.18%, and DAHS trained on one-tenth the data still outperforms it. DAHS trained on as few as 25 simulated shifts already outperforms both baselines at any training budget; we position this sample efficiency as the central contribution.

***Index Terms*—Dynamic programming, dynamic scheduling, heuristic algorithms, reinforcement learning, supervised learning.**

I. Introduction

Order dispatching on a warehouse floor is a sequential decision problem under uncertainty: orders arrive stochastically, each carries a due date and possibly a perishability constraint, and a small pool of pickers must be assigned work so as to minimise late and spoiled orders. In practice the decision is delegated to a *dispatching rule*, such as first-in-first-out or earliest-deadline-first, because rules are transparent, fast, and require no training. The well-known difficulty is that no single rule dominates: the rule that minimises lateness under light load is not the rule that does so when the queue is saturated or when a burst of perishable orders arrives. A controller that *selects* the rule appropriate to the current state, a *selection hyper-heuristic* [1], [2], can in principle capture the envelope of the pool without abandoning the operational advantages of rules.

The open question is how to *learn* the selector. The dominant modern answer is deep reinforcement learning (DRL) [3], which is attractive but sample-hungry and unstable on problems where the per-state advantage of one action over another is small relative to the return variance; that is exactly the regime of rule selection. Imitation learning [4] is a second answer. Both families face a structural limitation: DRL never sees the counterfactual cost of the rules it did *not* take, and imitation needs an expert demonstrator. The training signal we use has neither problem.

We take a different route. The cost of committing to a rule at a given state can be measured directly by simulation: fix the state, run each candidate rule forward for a short horizon, and record the cost it incurs. This is a rollout [5]. Rollouts are normally used online, re-run at every decision, which is too slow for a warehouse controller. Our approach runs rollouts offline, once, and uses them as a supervised training signal. For each state in a corpus of simulated shifts we roll out every rule, obtain a per-rule cost vector, and fit a supervised ranker to it. The expensive lookahead is thereby *distilled* into a cheap function approximator: at deployment the ranker is a single forward pass, and the rollouts live entirely in the training set.

This paper makes three contributions. First, *offline rollout distillation* as a training paradigm: truncated-horizon stochastic rollouts of a fixed rule pool, run once and offline, supply a per-state supervised target that avoids both the per-decision cost of online rollout and the instability of online reinforcement learning. Second, a *consistency result* (Proposition 1): the truncated-rollout cost is a consistent estimator of the full-horizon cost, with a bias that decays in the rollout horizon, which predicts, as confirmed empirically, that the horizon is the dominant design choice. Third, a *sample-efficiency result*: because the rollout supplies a directly measured, low-

variance per-state target, DAHS trained on 25 simulated shifts already outperforms a snapshot-trained ranker, an analytic lookahead controller, and a faithful offline reinforcement-learning baseline trained on ten times the data. We position sample efficiency, not the headline breach margin, as the contribution of record. An ablation shows the soft tempered-softmax label can be replaced by its hard arg-max with no measurable change, so the working mechanism is the rollout and its horizon, not the distributional objective.

II. Related Work

Selection hyper-heuristics: hyper-heuristics search the space of heuristics rather than the space of solutions [1]; the selection variant chooses, online, which low-level heuristic to apply, and is surveyed in [2]. Dispatching-rule selection for dynamic scheduling [6] is the closest application family. Prior selectors are typically trained by genetic programming, online reinforcement, or hard-label imitation, whereas DAHS trains a supervised ranker on rollout-derived per-rule cost vectors.

Rollout and approximate dynamic programming: a rollout policy improves a base policy by, at each state, simulating each candidate action followed by the base policy and committing to the action of least simulated cost [5]. Rollouts are a cornerstone of approximate dynamic programming [7] and are typically truncated to a finite horizon for tractability [8]. DAHS inverts the usual deployment: rather than re-running rollouts online at every decision, it runs them once, offline, to manufacture a supervised training set, encoding the per-rule cost vector as a soft label via a tempered softmax, a label-distribution representation [9]. Proposition 1 is a truncation-error argument in that tradition.

Learning-based dispatching: deep reinforcement learning has been applied to dynamic order picking [3], order batching [10], and warehouse scheduling [11], and its scalability for production scheduling remains under active investigation [12], [13]. We include a Proximal Policy Optimization [14] baseline at a matched budget and, separately, at a $60\times$ budget. Two recent learning-based selectors are the closest competitors: imitation learning of dispatching decisions [4], which trains on the actions of a single expert demonstrator, and offline reinforcement learning with maskable action-value learning [15], which fits a value function from logged data rather than regressing a directly measured cost. Section V compares against a faithful fitted-Q [16] instance of the latter, trained on the same logged shifts. A recent review [17] frames bootstrapped self-labelling as an emerging paradigm for machine learning in scheduling; DAHS is a theoretically grounded instance of it.

Warehouse operations and simulation: warehouse order-picking design and control has a long literature [18]–[21]. We evaluate in a discrete-interval simulator; the simulation modelling and its verification and validation follow standard methodology [22], [23]. No prior work, to our knowledge, combines truncated stochastic rollouts of a fixed rule pool as the source of an offline supervised training set, a calibrated supervised ranker as the deployed selector that sidesteps both the per-decision cost of online rollout and the instability of online reinforcement learning, and application to deadline- and perishability-constrained dynamic warehouse dispatching. DAHS occupies that intersection.

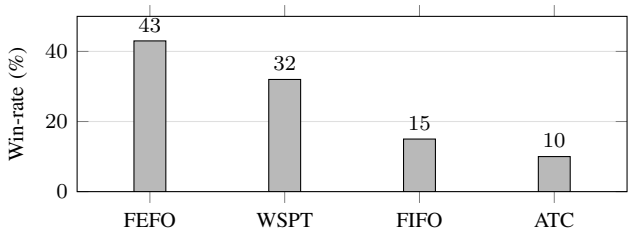

Fig. 1. Per-shift win-rate of each dispatching rule, the fraction of decision states at which it is the cost-minimising choice. No rule dominates.

III. Problem Setting and Method

A. The Dynamic Dispatching Problem

We consider a single warehouse shift of length T (8 hours). Orders arrive over the shift; order o has an arrival time a_o, a processing time p_o, an absolute due time d_o, a perishability flag, and a priority class. A fixed set of $m{=}10$ pickers processes orders, one at a time. Decisions are taken at the boundaries of $N{=}32$ equal intervals of 15 minutes; at each boundary the controller chooses a dispatching rule from a fixed pool, the chosen rule ranks the queue, and pickers are assigned greedily down that ranking. The operator's objective is a composite per-interval cost,

$$J = \frac{1}{N}\left(W_{\mathrm{b}}\, n_{\mathrm{b}} + W_{\mathrm{t}} \sum_{o} \tau_o + W_{\mathrm{u}}\, |Q|\right), \tag{1}$$

where n_{b} is the number of orders completed after their due time, $\tau_o = \max(f_o - d_o, 0)$ the tardiness, and $|Q|$ the count of orders left unprocessed at shift end. The weights $W_{\mathrm{b}}{=}3.0$, $W_{\mathrm{t}}{=}0.2$, $W_{\mathrm{u}}{=}0.005$ encode an operator who treats an outright SLA breach as the dominant cost; they are fixed before any learning and not tuned to the method. The environment is a deterministic discrete-interval simulator in which all stochastic quantities are pre-sampled from a single seeded generator, making a shift byte-identically reproducible from its seed, a property the rollout labeller requires. Arrivals follow a homogeneous Poisson process at 1.65 orders/minute; processing times and due-date offsets are triangular; a fixed 0.20 fraction of orders are perishable. The pool holds four rules: FIFO (by arrival), FEFO (first-expire-first-out), WSPT (weighted shortest processing time), and ATC (apparent tardiness cost). As Fig. 1 shows, all four win a non-trivial share of decisions, so selection is a real problem.

B. Rollout-Informed Training Labels

We simulate 250 training shifts under a round-robin behaviour policy, yielding $250 \times 32 = 8000$ decision states. The 25-dimensional state at each boundary summarises queue length and its lags, queue age, critical and perishable fractions,

labour utilisation, mean slack and dispersion, orders at near-term deadline risk, recent arrival rate, breach-rate lags, and temporal indices. For each state s_t we form a label over the four rules by truncated rollout: replay the shift to interval t, then for each rule h commit to h and apply the base policy for τ intervals, recording the cumulative cost $\hat{J}_h^\tau(s_t)$. The cost vector becomes a distribution by a tempered softmax,

$$p_h^\tau(s_t) = \frac{\exp(-\hat{J}_h^\tau(s_t)/\beta)}{\sum_{h'} \exp(-\hat{J}_{h'}^\tau(s_t)/\beta)}, \tag{2}$$

with β chosen once so the median label entropy falls in $[0.3, 0.7]$ ($\beta \approx 4.38$). When the perishable fraction is below 0.05 the FEFO mass is zeroed and renormalised. The deployed horizon is $\tau = 4$; Section V studies the choice.

C. A Consistency Result for Truncated Rollouts

Proposition 1 (Truncated-rollout consistency)**.** *Let $\bar{C}$ bound the composite cost in any single interval; such a bound is finite because queue capacity and picker count bound per-interval breach, tardiness, and unfinished count. Fix a state s_t with H_t intervals remaining. For rule h, let $J_h(s_t)$ be the full-horizon rollout cost and $\hat{J}_h^\tau(s_t)$ the τ-truncated cost, $\tau \le H_t$. Then*

$$0 \;\le\; J_h(s_t) - \hat{J}_h^\tau(s_t) \;\le\; (H_t - \tau)\,\bar{C} \;=:\; \Delta_\tau, \qquad \forall h,$$

and the truncated tempered-softmax label converges to the full-horizon label with $\mathrm{KL}(p^\infty(s_t) \,\|\, p^\tau(s_t)) \le 2\Delta_\tau/\beta$.

Proof sketch. The composite cost is a sum of non-negative per-interval contributions, so truncation removes a non-negative tail spanning $H_t - \tau$ intervals, each at most $\bar{C}$. Writing the label as a softmax of energies $-J_h/\beta$, the truncated energies differ by at most Δ_τ/β; the log-sum-exp normaliser is 1-Lipschitz in the supremum norm, so each log-probability shifts by at most $2\Delta_\tau/\beta$, giving the KL bound. □

Part one bounds the cost vector itself, so the consistency transfers to any label derived from it, including the hard argmax. The proposition has two testable consequences. The bias Δ_τ decreases monotonically in τ, so a ranker fit to longer-horizon labels fits a more internally consistent signal; the cross-validated soft cross-entropy indeed falls monotonically with τ. The bound governs only the bias, though: estimating $\hat{J}_h^\tau$ from finitely many stochastic rollouts also incurs variance that grows with τ. DAHS therefore faces a bias-variance trade-off in τ, with an interior optimum that Section V locates at $\tau = 3$.

D. Ranker, Regimes, and Controller

A Gaussian mixture fit to the training states (six components by BIC; mean pairwise adjusted Rand index 0.998 over ten refits) appends six regime-membership posteriors, giving the ranker 31 inputs. The ranker is a gradient-boosted decision-tree classifier [24] with four-class soft output, fit by inverse-entropy-weighted replication so the objective is the Kullback-Leibler divergence to the soft label. Hyperparameters are selected by 5-fold shift-grouped cross-validation (18 configurations; depth 4, 500 trees, learning rate 0.03; soft cross-entropy 0.677 against a uniform baseline of $\log 4 = 1.386$). Tree ensembles are not calibrated out of the box, so the output is post-processed with isotonic regression on a 20% held-out shift split, improving the expected calibration error from 0.063 to 0.028 and the Brier score from 0.130 to 0.107, clearing a pre-registered 0.05 acceptance threshold. A thin switching controller applies the FEFO mask, enforces a minimum dwell of $T_{\min}$=2 intervals to bound rule thrashing, and overrides the dwell when the ranker is highly confident. It is a stability guardrail, not a performance driver.

IV. Experimental Setup

All methods are evaluated on the same 50 held-out shift seeds, disjoint from the 250 training shifts. Beyond the default operating point, three scenarios stress the method: low load, balanced, and high-load-perishable; parameters are fixed before evaluation. We compare DAHS against the four static rules; *snapshot_xgb*, an ablation identical to DAHS but with the horizon collapsed to $\tau = 1$, which isolates the value of the rollout horizon; *greedy_mpc*, an independent analytic one-step-lookahead controller that simulates each rule for one interval and picks the cheapest; *LinUCB*, a contextual bandit; *PPO* [14] at a matched 8k-step budget (*ppo_fair*) and a 60× budget (*ppo_full*); and *offline_fqi*, a faithful offline reinforcement-learning competitor, fitted Q-iteration [16] with FEFO action masking, trained on the same 250 logged shifts under the same behaviour policy, reward, and gradient-boosted-tree model class as the DAHS ranker. The comparison therefore isolates the training signal, a directly measured per-rule cost vector versus a single bootstrapped value, from the function approximator. The primary KPI is the SLA-breach rate; we also report mean tardiness and the composite cost of (1). Uncertainty uses 10,000-resample bootstrap 95% confidence intervals; paired comparisons use the Wilcoxon signed-rank test with Benjamini-Hochberg control of the false discovery rate.

V. Results

A. Main Comparison

Table I reports every method on the default scenario. DAHS attains a 1.33% SLA-breach rate; the nearest competitors are greedy_mpc at 3.13% and the snapshot (τ=1) ranker at 3.73%. DAHS thus improves on the snapshot ablation by 2.40 percentage points: the multi-step rollout, absent from the snapshot, is doing real work. DAHS also attains the lowest composite cost. The offline reinforcement-learning baseline reaches 7.18%, behind even the snapshot ablation. DAHS is Pareto non-dominated; no baseline beats it on every metric simultaneously. FIFO clears orders aggressively for higher throughput, at more than four times DAHS's breach rate. Across scenarios (Table II) the picture holds: DAHS is rank one at balanced and default load. Under high-load-perishable saturation greedy_mpc edges DAHS on breach by 0.59 points, the one cell DAHS does not lead the breach metric, though

TABLE I
DEFAULT SCENARIO, 50 TEST SHIFTS. LOWER IS BETTER EXCEPT THROUGHPUT AND UTILISATION. THE PPO_FULL ROW COINCIDES WITH FEFO BECAUSE THE 500K-STEP POLICY COLLAPSES TO ALWAYS-FEFO.

Method	SLA breach	Tardiness	Cost	Util.
DAHS (ours)	**0.0133**	0.525	**3.09**	0.936
greedy_mpc	0.0313	1.822	9.19	0.846
snapshot_xgb (τ=1)	0.0373	1.589	8.77	0.851
ppo_fair (8k)	0.0385	0.257	3.92	0.970
FIFO	0.0660	0.618	7.57	0.983
LinUCB	0.0694	5.208	23.61	0.771
offline_fqi	0.0718	0.531	7.46	0.960
WSPT	0.0949	10.671	43.56	0.686
FEFO / ppo_full (500k)	0.1181	0.997	12.60	0.947
ATC	0.1572	1.238	16.37	0.940

TABLE II
SLA-BREACH RATE BY SCENARIO (50 TEST SHIFTS).

Scenario	DAHS	greedy_mpc	snapshot	offline_fqi
low load	0.0000	0.0000	0.0000	0.0000
balanced	**0.00039**	0.00546	0.00576	0.00342
default	**0.0133**	0.0313	0.0373	0.0718
high-load-perish.	0.1943	**0.1884**	0.1949	0.6192

DAHS keeps the lower composite cost there; offline_fqi instead collapses to a 61.9% breach rate, more than three times DAHS.

B. Sample Efficiency (the Central Result)

The 2.40-point breach margin is, on its own, a modest empirical win. The result we ask the reader to weight is how little data DAHS needs to reach it. We retrain DAHS from scratch on budgets of 25 to 250 shifts (five replications each below 250) and evaluate on the same 50 test shifts. Fig. 2 plots the outcome. DAHS trained on 25 shifts attains a 1.44% breach rate, already far below the snapshot ranker (3.73%) and greedy_mpc (3.13%). The curve is essentially flat from 25 to 250 shifts, so roughly 90% of the training corpus is redundant. This is the operational signature of rollout distillation: every training state carries a directly measured per-rule target rather than the noisy, shift-level return an RL agent must learn from, so the supervised signal is dense and low-variance and the ranker saturates its learnable structure within a few dozen shifts. The offline_fqi baseline, by contrast, improves with budget but stays far above DAHS and is still descending at 250 shifts (11.55% at 25, 7.18% at 250) with a cross-replication standard deviation of 4.5 points at 25 shifts against 0.3 for DAHS. DAHS from 25 shifts beats offline_fqi from the full 250 by 5.7 points: a deployable selector from one-tenth the data.

C. Rollout Horizon, Robustness, and Ablations

Fig. 3 and Table III confirm the bias-variance trade-off of Section III-C. As τ rises from 1 to 4 the cross-validated soft cross-entropy falls monotonically (0.817, 0.764, 0.709, 0.677), the bias term; the operational breach rate, however, falls steeply to τ=3 (1.05%) and rises slightly at τ=4 (1.33%),

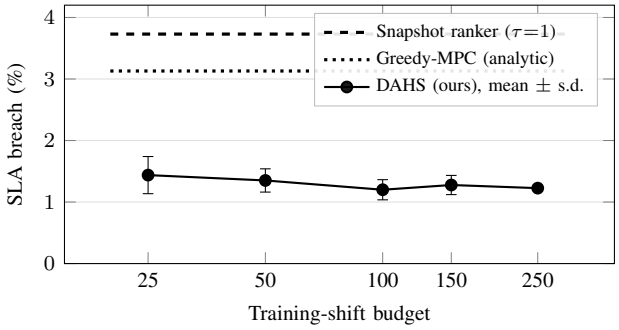

Fig. 2. Sample efficiency. DAHS SLA-breach rate versus the number of simulated training shifts (mean ± standard deviation over five replications; zero at 250 because all replications draw the identical full corpus). DAHS trained on 25 shifts already sits well below both reference baselines.

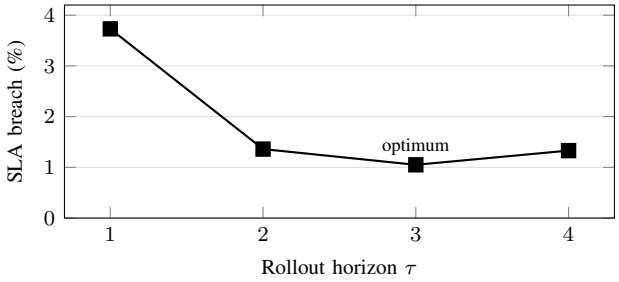

Fig. 3. SLA-breach rate versus rollout horizon. The U-shape, a steep improvement to τ=3 then a slight regression at τ=4, is the bias-variance trade-off of Proposition 1.

the variance term. The optimum is interior at $\tau = 3$. Even the worst non-trivial horizon, the τ=1 snapshot, is 3.5× worse than τ=3: the rollout horizon is the single most important design choice. Across a 12-cell robustness grid (four arrival rates crossed with three deadline-tightness levels, only one calibrated) the method ranking is stable; DAHS is no worse than the snapshot ranker in all 12 cells, no worse than greedy_mpc in 10 of 12, and no worse than offline_fqi in all 12, by a margin that widens with load. Degradation under heavier load is graceful and never collapses, whereas the best static rule degrades to 70% breach in the hardest cell.

Three ablations isolate single components. Replacing the soft tempered-softmax label with its hard arg-max leaves the breach rate statistically unchanged (1.27% versus 1.33%; $p = 0.11$) and the composite cost marginally lower, so the distributional form of the label is not a source of DAHS's performance; the rollout and its horizon are. Removing isotonic calibration degrades the breach rate to 2.94% and the cost to 7.85 ($p < 0.001$), because the controller's entropy gate acts on the predicted probabilities. Removing the switching controller slightly *improves* KPIs (breach 1.18%, cost 2.74); we retain it deliberately, because its role is to bound switching and enforce the perishability constraint, not to win the comparison. A Shapley-value analysis [25] attributes the ranker's decisions to operationally sensible features: queue length and its lags, mean slack, near-term deadline risk, and the interval index.

TABLE III
ROLLOUT-HORIZON SENSITIVITY (50 TEST SHIFTS). THE CROSS-VALIDATED SOFT CROSS-ENTROPY MEASURES LABEL FIT; LOWER IS BETTER THROUGHOUT.

τ	CV soft cross-entropy	SLA breach	Composite cost
1 (snapshot)	0.817	0.0373	8.77
2	0.764	0.0136	2.72
3	0.709	**0.0105**	**2.31**
4 (deployed)	0.677	0.0133	3.09

D. Reinforcement-Learning Baselines and Real-Data Grounding

Whether the deep-RL baseline is under-trained is a fair question; the evidence says the issue is structural. At 8k steps PPO oscillates between two rules (3.85% breach); a $60\times$ budget makes matters worse, collapsing onto always-FEFO (11.81%). The per-state advantage of one rule over another is small relative to the shift-return variance, so the policy drifts to the highest unconditional-return rule. The offline baseline does not collapse at the default operating point: it ties DAHS on tardiness (0.531 versus 0.525) and beats the snapshot and analytic controllers on composite cost, yet loses the primary metric by 5.85 points (95% CI [3.85, 8.11]). An SLA breach is a rare, expensive event, and a scalar bootstrapped value smears that signal across the bulk cost of tardiness and queue volume; the per-rule rollout-cost vector measures the breach-laden cost directly. We also validated the simulator's input distributions against the Olist Brazilian e-commerce trace [26]: the due-date window matches well (Kolmogorov-Smirnov D=0.039, normalised Wasserstein 0.036), while the real inter-arrival stream is far burstier than Poisson (D=0.153, coefficient of variation 2.68 versus 1.00, skewness 11.0 versus 2.0). Re-running the full comparison with the frozen DAHS ranker under a bootstrap of the empirical bursty arrivals, every method degrades, but DAHS retains rank one and its paired margin over the snapshot ranker widens to 2.50 points (95% CI [1.68, 3.45]). The advantage is therefore not an artefact of the idealised arrival process.

VI. DISCUSSION AND CONCLUSION

The results support a narrow but well-grounded claim. On deadline-constrained warehouse dispatching, a selector trained by offline rollout distillation is sample-efficient, theoretically consistent in its training signal, and robust across untuned operating points and a realistically bursty arrival stream. The headline SLA-breach margin over the strongest learned baseline is real but modest; we have been explicit that the contribution is the mechanism and its data efficiency, not the size of that margin. DAHS helps most where rule selection is genuinely contested: moderate-to-high load with tight deadlines, where the queue state swings the best rule from interval to interval. It helps least at the extremes, where every rule is adequate or the rules converge under saturation. Two deployment properties reinforce the operational case. DAHS runs no simulation at deployment, paying the lookahead cost once, offline, at labelling time, whereas the analytic controller simulates every rule at every decision. And the action it emits is one of four named rules a supervisor already understands, rather than an opaque assignment; we scope this carefully, since the selector itself is a tree ensemble interpreted only post hoc.

The broader methodological point is that an expensive online computation, the rollout, can be paid once, offline, and amortised into a cheap learned approximation, provided the offline computation becomes a supervised target rather than a reinforcement signal. A controlled comparison supports the proviso: an offline reinforcement-learning baseline given the same logged shifts, the same model class, and a fair hyperparameter search, but trained to bootstrap a value, loses the primary metric by 5.85 points. Limitations remain. Evaluation is simulation-only, and pick time has no public real-world analogue; the simulator was calibrated at one operating point, though the ranking is stable across eleven further untuned cells; DAHS concedes the breach metric in the single saturation cell; and a modern conservative offline-RL method (CQL, IQL) is left to future work, alongside evaluation on a logged real-warehouse trace and an online variant that adapts the rollout budget to the remaining horizon.

REFERENCES

[1] J. H. Drake, A. Kheiri, E. Özcan, and E. K. Burke, "Recent advances in selection hyper-heuristics," *European Journal of Operational Research*, vol. 285, no. 2, pp. 405–428, 2020.

[2] T. Dokeroglu, T. Kucukyilmaz, and E.-G. Talbi, "Hyper-heuristics: A survey and taxonomy," *Computers & Industrial Engineering*, 2024.

[3] S. Mahmoudinazlou, A. Sobhanan, H. Charkhgard, A. Eshragh, and G. Dunn, "Deep reinforcement learning for dynamic order picking in warehouses," *Computers & Operations Research*, vol. 182, p. 107112, 2025.

[4] Han and Jung, "Imitation learning of dispatching policies for dynamic scheduling," *Computers & Industrial Engineering*, 2025.

[5] D. P. Bertsekas, *Rollout, Policy Iteration, and Distributed Reinforcement Learning*. Athena Scientific, 2020.

[6] M. Durašević and D. Jakobović, "Heuristic and metaheuristic methods for dispatching-rule selection in dynamic scheduling," *Engineering Applications of Artificial Intelligence*, 2022.

[7] D. Simchi-Levi, R. Sun, and Y. Zhao, "Approximate dynamic programming for dynamic operations problems," *Operations Research*, 2021.

[8] Y. He, T. Charalambous, and P. A. Stavrou, "Truncated rollout for approximate dynamic programming," *arXiv preprint*, 2024.

[9] X. Geng, "Label distribution learning," *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 7, pp. 1734–1748, 2016.

[10] L. Cheng *et al.*, "A deep reinforcement learning hyper-heuristic for order batching," *Applied Intelligence*, 2024.

[11] Y. Zhang *et al.*, "An LSTM-based PPO approach for warehouse order scheduling," *Computers & Industrial Engineering*, 2024.

[12] P. Stöckermann *et al.*, "On the scalability of deep reinforcement learning for production scheduling," *arXiv preprint*, 2025.

[13] P. Tassel *et al.*, "A reinforcement learning environment for job-shop scheduling," *Machine Learning and Knowledge Extraction*, 2023.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," in *arXiv preprint arXiv:1707.06347*, 2017.

[15] Pluijm *et al.*, "Offline-LD: Offline reinforcement learning with maskable Q-learning for scheduling," *arXiv preprint*, 2025.

[16] D. Ernst, P. Geurts, and L. Wehenkel, "Tree-based batch mode reinforcement learning," *Journal of Machine Learning Research*, vol. 6, pp. 503–556, 2005.

[17] Sauer *et al.*, "Machine learning for scheduling: A paradigm shift," *arXiv preprint arXiv:2512.22642*, 2025.

[18] R. de Koster, T. Le-Duc, and K. J. Roodbergen, "Design and control of warehouse order picking: A literature review," *European Journal of Operational Research*, vol. 182, no. 2, pp. 481–501, 2007.
[19] K. J. Roodbergen and I. F. A. Vis, "A model for warehouse layout," *IIE Transactions*, vol. 38, no. 10, pp. 799–811, 2006.
[20] N. Boysen, R. de Koster, and F. Weidinger, "Warehousing in the e-commerce era: A survey," *European Journal of Operational Research*, vol. 277, no. 2, pp. 396–411, 2019.
[21] N. Boysen, S. Emde, M. Hoeck, and A. Scholz, "Order dispatching in modern warehouses: A survey," *European Journal of Operational Research*, 2025.
[22] A. M. Law and W. D. Kelton, *Simulation Modeling and Analysis*, 3rd ed. McGraw-Hill, 2000.
[23] R. G. Sargent, "Verification and validation of simulation models," in *Proceedings of the Winter Simulation Conference*, 2013.
[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.
[25] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
[26] Olist, "Brazilian e-commerce public dataset by olist," Kaggle, 2018, https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce.